

Adaptive Demand Based Associativity with Evicted Address Filtering

Yash Ajay Deshpande, Dhruv Bittal Singhi and Austin James

ECE752 - Advanced Computer Architecture - I, Prof. Guri Sohi

Abstract

The ever widening “memory wall” has necessitated improved cache design and policies. Unlike primary L1 caches, secondary caches experience only a sparsified, filtered access stream but with a relaxed latency constraint. Contemporary organizations are set associative, providing low tag comparison overhead but are bound by local replacement. Contemporary secondary caches demand high hit rates but due to the high energy and latency requirements of fully associative design, are limited to a fixed associativity and local replacement. To this extent, this work addresses cache thrashing and pollution through a (1) Variable Way (V-Way) Cache that provides a global replacement policy and dynamic associativity, and (2) Evicted Address Filtering (EAF), that uses an auxiliary Bloom-filter that remembers and improves the insertion through reusability prediction. We also implement and present the results for a novel design combining both the ideas.

We use Intel’s Pin, a dynamic instrumentation tool for our design simulations. Our results demonstrate, VWay emulates or is better generally across all SPEC2017 benchmarks than a baseline Least Recently Used (LRU) implementation for the same associativity and capacity. While EAF doesn’t always guarantee reduced miss rates, it shows visible improvements in non LRU friendly workloads. We report an average miss rate reduction of 21.02% using the V-way cache and 9.70% for the EAF over a baseline 4-way LRU policy cache for 16 SPEC CPU2017 benchmarks. Our combined implementation yields a higher miss-rate reduction of 21.74%. These results highlight that optimal cache insertion and eviction based on global replacement and reuse frequency significantly improve performance at moderate hardware complexity.

Index Terms

Variable-Way cache, Evicted-Address Filter, Dynamic associativity, Global replacement, Cache thrashing and pollution, Bloom-filter

I. INTRODUCTION

IN a set associative cache, the number of entries visible to a replacement policy is limited to the specific cache set. If we could increase the visibility of the replacement policy to look at the global access pattern, a much better decision regarding eviction could be made. However, this also means that on accessing a particular set, the

corresponding data block could be anywhere in the data store and not just confined to that set. Another characteristic that we require is storing only blocks that have high temporal reuse i.e. blocks that are likely to be accessed multiple times within a short time interval. However, two problems degrade this, first, cache blocks with little or no reuse can evict blocks with high reuse from the cache. This problem is referred to as cache pollution. Second, when there are a large number of blocks with high reuse, they start evicting each other from the cache due to lack of space in the same set even when other sets may have vacant unused space. This problem is referred to as cache thrashing. Both pollution and thrashing increase the miss rate of the cache and consequently reduce system performance[3][4][5].

A. *Work Similar to the V-way Cache*

Several other techniques have been proposed that try to get the advantages of higher associativity while avoiding the same level of overhead. Further, several techniques have been seen in literature that track reuse behaviour of data to decide inclusion or exclusion on the cache. Some of them are:

1) **Indirect Indexed Cache:** Using software management and indirection, this cache structure tries to provide full associativity theoretically. The cache employs collision tables to do this. Collision tables are direct mapped with entries containing a tag, a data pointer and a pointer to the next level in the chain. On a cache access, we index into the collision table and match tags. If the tag matches we use the data pointer to get the data, else we move to the next node. The flow is similar to accessing a linked list. If no tag match is found, till the end of the list, it's a cache miss. Eviction and other management is handled by software. Frequently accessed nodes move up the list, while less reused ones move down. Ideally collision tables could span over the entire cache but practically their size is limited. This algorithm thus combines reference recency and frequency information with a hysteresis effect, while relying only on block reference bits and periodic data-structure updates[12]. However, the drawbacks involve variable access latency and complex software management.

2) **Scavenger Cache:** Divides the conventional cache into two halves, a victim cache and a half sized conventional cache. Data evicted from the conventional cache is inserted into the victim cache depending on reuse behaviour tracked using bloom filters. This is an LLC technique.

3) **ZCache:** This technique provides full associativity, by trying to ensure that every replacement replaces an empty victim block, the accuracy of which depends on a hashing function. Each block gets mapped to multiple sets, if either one of them matches it's a hit. If any of them is empty insert the block there, else pick a block to 'move' and insert the new block. This moved block now uses its own hashing function to find a place to move to and the process cascades. The replacement happens in parallel with higher level cache access and therefore does not incur additional latency.[13]

4) **NuRAPID Cache:** This cache uses distance replacement algorithm i.e the promotion and demotion of data blocks to different distance groups. Forward pointers map tag stores to data blocks while reverse pointers map

data blocks back to the tags. The rest of the cache structure remains the same. The main idea is that not all data needs to be accessed at high speed with reused blocks kept physically closer while low reuse blocks are kept away. Replacement is performed at the furthest distance and the structure allows full associativity.

5) **Group Associative Cache:** This cache structure can be used for L1 caches as well. It starts as a direct mapped cache. When blocks are evicted, a decision is made whether to evict them completely or insert them at another location called a hole within the cache. Cache management therefore comes down to three main steps. Identifying if blocks must be evicted or not, identifying presence of holes in the cache, locating blocks displaced to holes. These are performed by structures such as Set-reference History Table (SHT) and the Out of Position or OUT directory.

B. Prior works to EAF and their shortcomings

The following policies are prior works that aims to address either pollution or thrashing and rely on coarser granularity predictors:

1) **Thread-Aware Dynamic Insertion Policy:** TA-DIP employs set dueling to choose on a per thread basis, between conventional LRU insertion and the thrash resistant Bimodal Insertion Policy (BIP). While this suppresses thrashing when threads have dissimilar working set sizes, TA-DIP has no notion of block level reuse. Naturally, low reuse blocks may still displace high reuse blocks when the chosen policy is LRU, and, conversely, BIP can demote genuinely high reuse blocks to the LRU position, inflating miss rate whenever thrashing is absent.

2) **Thread-Aware Dynamic Re-Reference Interval Prediction:** TA-DRIP improves on TA-DIP by pairing set-dueling with the Round Robin (RRIP) replacement policy, which inserts with an intermediate priority and therefore curbs pollution relative to LRU. However, like TA-DIP it works only at thread granularity and learns reuse after a block resides in the cache[8], leaving it vulnerable to both mispredictions and thrashing in mixed-reuse workloads.

3) **Signature-Based Hit Prediction:** SHIP clusters blocks by the program counter (PC) responsible for the miss and updates a per-signature saturating counter. Blocks from signatures deemed “high-reuse” enter at the MRU position, while others are demoted. The underlying assumption that all blocks generated by the same PC share identical reuse behaviour often breaks down, leading to false positives. Moreover, when the set of “high-reuse” signatures exceeds cache capacity SHIP has no mechanism to reduce the resulting thrashing.

4) **Run-Time Cache Bypassing:** RTB labels misses using 1 KB memory region identifiers and bypasses the cache when the incoming block’s predicted reuse is lower than that of the would be victim. RTB therefore inherits SHIP’s coarse granularity and suffers the same generalisation errors and inability to mitigate thrashing once many regions are classified as high reuse.

5) **Miss Classification Table:** MCT retains only the most recently evicted tag per set to distinguish conflict(if tracked tag matches with missed tag) from capacity misses. A single tracked tag is inadequate for modern, highly

associative last-level caches; many conflict misses are mis-classified as capacity misses, especially in multi-core systems. Naïvely enlarging MCT to keep additional tags increases storage cost and can itself trigger thrashing when more blocks are predicted as “conflict” than the cache can accommodate.

Evicted Address Filter augments the cache with a single Bloom filter backed structure that records the addresses of recently evicted blocks[1]. Reuse prediction is therefore per-block rather than per-thread, program counter or memory-region. By clearing the filter globally, even a pathological hot loop can only saturate MRU status for a bounded window (total number of blocks); thereafter it must earn new MRU slots gradually while existing ones age out. That bounded occupancy is what reins in thrashing without sacrificing pollution protection.

II. V-WAY CACHE

A solution to the problem of cache thrashing and identification of a global victim for replacement is provided by the V-way cache. The V-way cache dynamically varies the per-set associativity in response to the application demand. Thrashing is minimized by allowing sets to use vacant space in other sets. Further, replacement decisions are made on the basis of global cache reuse pattern rather than local set information. This is achieved by modifying the cache structure to have double the number of tag-store entries as data-stores, using serialized access with pointer metadata to locate data and employing a practical global reuse replacement policy. Doubling the number of tag-store entries is relatively inexpensive, typically adding 2-3% to the overall storage requirements of a secondary cache.[14]

A. *Structure*

The structure of the V-way cache scaled for illustration purposes is shown in Fig 1. As can be seen from the figure, the number of tag-store entries is double compared to the number of data-store entries. In addition to status and tag bits as in a conventional set associative cache, we have a pointer metadata called the Forward Pointer (FPTR) that locates the corresponding data-store entry for that tag-store entry. Similarly, a Reverse Pointer (RPTR) is maintained with a data-store that locates the tag corresponding to that data - block. These pointers are required as the V-way cache, to achieve dynamic associativity breaks the one - to - one mapping of sets to data. Every tag and data store pair must always point to each other by using a FPTR-RPTR pair. Further, to implement the global reuse replacement policy, 2-bit saturating reuse counters are maintained with every data store entry.

B. *Operation*

Fig 1 also illustrates the operation of the V-way cache. A cache access consists of one or two sequential lookups depending whether the access is a hit or a miss. A hit is determined by matching tags as in a conventional cache. Referring to 1 in Fig 1 on a tag match, the FPTR from the correct way must be selected to access the data-store. Therefore the data-store access cannot be performed in parallel with the tag-store access. This delay is acceptable

for large caches that are already slow, also significant energy savings are obtained in serial access. Once the FPTR is obtained, the data from the data-store is located and its reuse counter is incremented. In addition to the global replacement policy, the V-way cache also maintains LRU counters that must be updated to reflect local set access history. On a miss, either operation indicated by 2a or 2b takes place depending on whether the set has empty/invalid tags or not. A miss always identifies a data-victim from the data-store and a tag victim from the tag-store. The operation is in accordance with 2a if the set has invalid tag stores. In this case the global replacement policy is invoked. The first invalid tag found in the present set is selected for storing the new tag value for which the miss occurred. A pointer (PTR) that was initialized to the beginning of the data-store array, tests and decrements the reuse counters till a reuse counter with counter value 0 is found. The pointer is cyclic and will restart at the beginning of the array if no victim is found in the first pass. The data-store entry corresponding to the counter 0 value is known as the data-victim. The current reverse pointer of this data-victim is known as the tag-victim. This tag-victim in the tag-store array is invalidated. This is essential to avoid multiple tags pointing to the same data-store and is required as the V-way cache can increase the associativity of a set on demand only if another set loses this associativity. All sets cannot have higher associativity simultaneously. This is acceptable as only certain sets are more frequently accessed than others. Moreover, this skewed demand often appears in phases in a program. The FPTR of the tag-store where the new tag value was added is updated to point to the data-victim. The new data is added to the location of the current data-store and its RPTR is updated to point to the newly validated tag-store entry. The PTR then increments by one, to allow the newly added data to be reused before being tested for replacement. In case the current set does not have an invalid tag, the V-way cache resorts to a local LRU replacement policy. Global reuse replacement cannot be used here as either 2 cache-lines would have to be invalidated to accommodate one new line, or the hashing of addresses into tag-stores would have to be modified. The tag-victim in this case is found by the local LRU replacement; this tag victim consists of a valid FPTR. In the Fig 1 2b indicates this. The data-line to which it points is known as the data-victim. The RPTR also correctly points back to the tag-store and need not be updated. In both cases of a miss, the LRU counters are suitably updated.

C. Practicality of Global Reuse Replacement

The method of reuse replacement described above is guaranteed to find a victim but it suggests a variable replacement latency. In the theoretical worst case scenario all counters can already be saturated, in which case it would take $(2^N - 1) \times \text{NUM_REUSE_COUNTERS}$ decrement operation to find the data victim for a N bit saturating reuse counter. This worst case theoretical maximum however is unlikely to occur. This is because most cache lines in higher level caches experience little reuse. For example, in applications with high spatial locality, data is often easily found in the same cache block. While the primary L1 cache accesses will have high reuse for the block, it is accessed only once in the L2. Also, due to decoupling of the tag and data-stores, it is rare to find

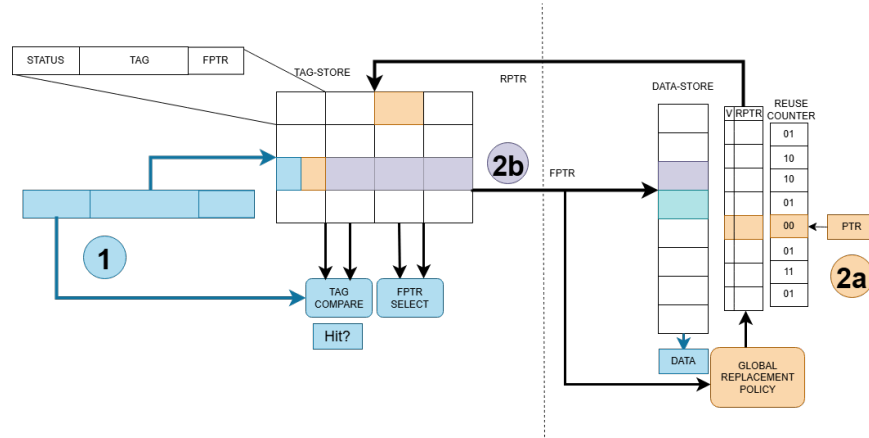


Fig. 1: V-Way cache organization.

a stride based memory access pattern generating long victim distances. Further the global replacement is invoked only when there is a cache miss, this means finding the data victim can take place in parallel with the high latency operation of accessing higher levels of memory. On the circuit implementation level, testing a counter for zero, is NOR operation of all its bits. Multiple counters can be tested in parallel, with a priority decoder to select the first counter that has become 0.

III. EVICTED ADDRESS FILTER

The objective is to predict the reuse behavior of a missed cache block based on its own past behavior. The idea comes from the observation that to distinguish the reusability of a block, it is sufficient to keep track of a set of recently evicted blocks[2]. Depending on the replacement policy used by the cache, the presence of a bit in the EAF indicates that the block should be placed with high priority at the MRU position (for the conventional LRU replacement policy); otherwise, it is inserted with low priority, for example, at the LRU position. The size of the EAF, that is, the number of recently evicted blocks it can track, is set to match the cache size, covering the worst-case scenario in which all blocks are high-reuse. Thus, any block with a large reuse distance is unlikely to be present in the EAF, and any block that is prematurely evicted will be stored in the EAF. Because the EAF operates only on cache misses, it does not affect the cache-hit critical path.

A. Working-Set Size

1) **Case 1: Working-set $\geq$ Cache + EAF:** All missed blocks are predicted to be low reuse and inserted with low priority, leading to under-utilization of the cache because no blocks receive high priority.

2) **Case 2: Working-set $\leq$ Cache + EAF:** Every missed block's address is present in the EAF; all blocks are predicted to be high reuse and inserted with high priority, causing cache thrashing.

Case 1 can be handled by using a Bimodal Insertion Policy (BIP) for blocks predicted to have low reuse, instead of always inserting them with low priority. BIP inserts a small fraction of blocks with high priority and the remaining blocks with low priority, helping to keep a large working set in the cache[6]. Cache thrashing can be mitigated by restricting the total number of blocks predicted to have high reuse. This can be done fairly if every block is periodically required to prove its reusability[7], which an appropriately configured Bloom filter can achieve. Fig 2 below illustrates the EAF mechanism.

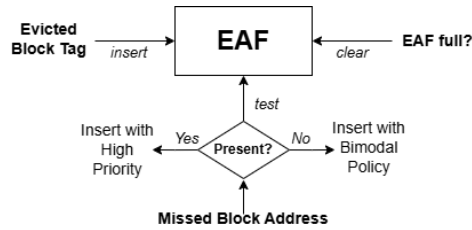


Fig. 2: EAF Mechanism.

B. Bloom Filter

A Bloom filter is a probabilistic data structure that stores a compressed representation of a data set and supports three operations: insert, query, and delete. The filter is a bit array of size m with k hash functions. Initially, the filter is all zeros. To insert an element, the k hash functions are applied to the element to obtain k indices in the bit array, and the bits at these indices are set to one. To query an element, the k hash functions are applied; if all bits at the resulting indices are one, the element is considered present. If any bit is zero, the element is definitely not present.

The filter can produce false positives: it may indicate that an element is present when it is not, because different elements can map to the same bits. However, it produces no false negatives; if the filter indicates that an element is absent, it is definitely absent. The false-positive rate can be tuned by adjusting the Bloom-filter parameter α . Because the Bloom filter does not store complete tags, it consumes relatively little storage, and indexed lookups into a bit array are more energy-efficient than associative lookups. Fig 3.a below shows the insert and query operations of the Bloom filter.

A counter tracks each insertion into the EAF, incrementing on every insert. When the EAF is full and a new block is inserted, the counter resets to zero. If the counter exceeds a threshold, the EAF is reset and all blocks are predicted to be low-reuse. This prevents the EAF from growing too large and keeps the cache from thrashing. Fig 3.b below depicts the complete EAF with the Bloom filter and counter.

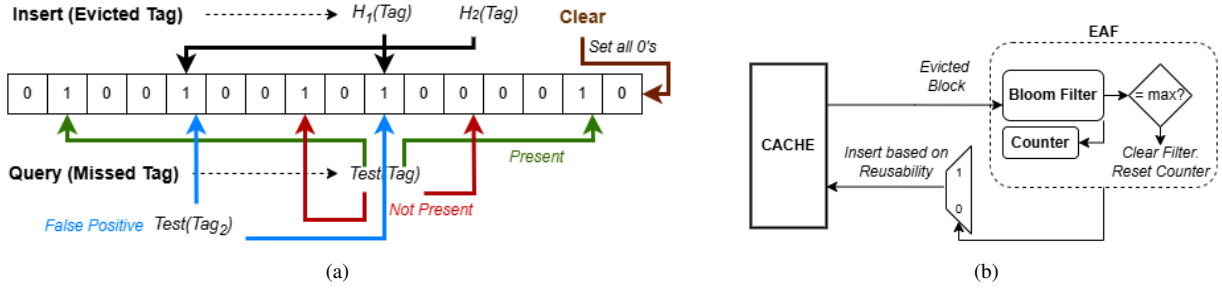


Fig. 3: (a) Bloom Filter Mechanism and (b) Organization of EAF and Cache

C. Hardware cost

Because the EAF is an external addition to the cache, it requires no changes to the cache structure. The EAF comprises three microarchitectural structures: (1) a Bloom filter bit array, (2) a counter that counts up to the number of blocks in the cache, and (3) a multiplexer-cum-logic block that selects the insertion policy. The main storage is the bit array, whose size equals the number of cache blocks C multiplied by α , the number of bits per address stored in the Bloom filter. If each cache block has size B and each tag entry has size T , then the relative storage overhead is given by:

$$\frac{\text{Bloom-filter size}}{\text{Cache size}} = \frac{\alpha C}{(T + B) C} = \frac{\alpha}{T + B}$$

The Bloom filter's false-positive probability is

$$p = 2^{-\alpha \ln 2}.$$

The parameter α trades off false-positive rate against storage overhead. As our results show, reducing false positives saturates once $\alpha \geq 8$ [11]. For a 256 KB L2 cache with 64-byte blocks, the Bloom filter consumes only 1.51% of the cache size. Our results later demonstrate that this additional storage is offset by performance gain averaging upto 9.70% over the baseline four-way LRU cache.

IV. INTEL PIN IMPLEMENTATION

We simulated & tested our implementations using Intel Pin, a dynamic binary instrumentation (DBI) framework for the IA-32 and x86-64 instruction-set architectures. It enables the creation of dynamic program analysis tools. The process of modifying the instructions of a binary program while it executes is Dynamic binary instrumentation. The tools created using Pin, called Pintools, can be used to perform program analysis on user space applications. As a DBI tool, instrumentation is performed at run time on the compiled binary files. Thus, it requires no recompiling of source code and can support instrumenting programs that dynamically generate code[15]. This provided us with:

- Ability to benchmark our implementations using the actual binaries of the SPEC2017 benchmark suite

- Ability to simulate a large number of memory accesses as needed to properly test multilevel cache hierarchies without running into instruction count limitations faced by trace-based simulators.
- Ability to work only with the cache design under concern, without analysing CPU requirements, actual memory latencies and cycle accurate timing information.

A. *Helper Functions of Intel Pin*

To track memory accesses, we must check each instruction within the binary if it is a memory operation. To instrument each instruction in a program binary, Intel Pin provides a function called *Instruction*. It is used to simulate instruction and data cache access behavior during the execution of a program. Memory-related instructions in a binary are routed to corresponding cache tracking functions. For memory accesses, we used the *INS_InsertPredicatedCall* function which ensures the instrumentation occurs only if the predicate condition of the instruction is true. The data cache instrumentation is performed using the function **MemRefSingle** for memory accesses smaller than 4 bytes and *MemRefMulti* for larger ones. Instruction cache instrumentation is performed before the instruction in the program binary executes using a function called *InsRef*

Both the EAF & the V-Way cache as well as its combined implementation are suitable replacement policies for higher level caches as they access large data structures to make a replacement decision. L1 caches require very low access latency due to their very high access rate. Hence we tested our implementations on level-2 caches, while keeping the L1 cache constant.

B. *Snippets of Important Pieces of Code*

Intel Pin has a builtin tool to simulate multilevel cache hierarchy called *allcache.cpp*. The standard *allcache* simulates 3 levels of cache. This built-in structure uses a round-robin cache replacement policy. Thus, we started by building a similar pin tool with 2 levels of cache as needed. We then implemented a Least Recently Used cache replacement algorithm to use as our baseline. The code was designed to be modular & consistent to allow for easy reuse of the same structure when evaluating different replacement policies. The cache class has a template parameter called “SET” using which the same class can be used to model different replacement policies dynamically.

Template declaration of the CACHE class

```
template< class SET, UINT32 MAX_SETS, UINT32 STORE_ALLOCATION >
class CACHE : public CACHE_BASE
```

This proved to be greatly useful when testing in the later stages of the project.

The LRU replacement policy was implemented using 2 methodologies:

- Using LRU counters per each way in the set (emulating hardware LRU implementation)
- Using hash maps & doubly linked lists.

1) **LRU Implementation:** Based on initial testing, we used counter-based implementation to simplify adding the V-way cache on top. This implementation assigns counter values ranging from 0 to associativity-1 to each of the ways within a cache set. The counter value of the block that is accessed is set to the maximum while the counter value of other ways is decremented. The cache way with the least counter value i.e. 0 is selected for replacement.

2) **V-way Implementation:** Intel Pin's built-in cache simulation classes do not support the decoupled tag & data store access required for the V-way cache architecture. So, we designed custom classes for the data store and tag store as well as classes to store set meta data like reuse counters. All these data structures were initialized dynamically inside an independent class for the V-way cache. Parameters like cache size, line size, associativity & tags-to-data ratio were passed as arguments to the constructor. For every single memory access, initially the *Find()* function is called. On a cache hit, the *Find()* function updates the LRU counter of the accessed block to the maximum current associativity & reduces the counter values of the other ways.

On a cache miss, the *Find()* returns a zero, & the *Replace()* function gets called. Initially it checks if the associativity of the current set is less than or equal to the max associativity. If the current associativity is less than maximum then the *global reuse replacement* policy gets invoked. It operates in the following steps:

- Validate a new way within the set to store the new tag value & increase the current associativity of that cache set.
- Find an entry within the data store to save the cache data block.
- Update the forward pointer FPTR to point to the data store entry.
- Set the return pointer RPTR to point at the correct tag store entry.
- If the data store entry was previously pointing to a valid tag store then invalidate it & decrement the current associativity of that cache set.

If the current associativity of a set is equal to the maximum, then replacement is done using the standard LRU policy. Based, on whether the *Replace()* function gets called, the appropriate counter is updated to keep a count of the number of cache hits & misses.

3) **EAF Implementation:** For implementing the Evicted Address Filter, we maintain a parameterizable value of α . This is the number of bits needed to store a single tag address inside the bloom filter. The size of the bloom filter itself is a multiple of the α & the total number of blocks that the cache can fit. Upon a cache eviction, we record the tag address inside the bloom filter using the H3 class of hash functions[10] implementation of which is mentioned in the [Appendix A]. Upon accessing a new cache block, its tag is checked against the bloom filter using the same hash functions. The priority of the cache blocks is maintained using the LRU counters as implemented in the baseline policy.

4) **Combined Implementation:** The impact of the EAF on the V-way cache is noticeable only if it is accessed a sufficient number of times. We tried different ways of adding the EAF on top of the V-way cache & tested each

for its impact on the results. Here we present the observations from our implementation where the EAF augments the V-way cache while using the global reuse replacement policy as this configuration yielded the most compelling results.

The EAF does not require modifications to existing cache structures. It is a simple extension that can inform the existing replacement policy about the block's historical access pattern thus helping it make a smarter replacement decision. On every cache miss, the evicted tag address gets recorded in the EAF using the mechanisms previously explained. And we check the EAF bloom filter for any block before placing it in the desired set. The output of the filter is recorded as the priority of the block. If the tag address is present inside the EAF, then it is placed at the highest priority in the set. However, if the tag address is not present in the EAF, then it is placed in the set with the least priority. This is managed by setting the LRU counters appropriately. For every block that is placed in the cache, an older block is invalidated using the data store victim mechanism of the V-way cache.

V. EXPERIMENTAL METHODOLOGY

Our baseline cache configuration is detailed in Table I. As the L2 cache gets accessed only when there is a miss in the L1 cache, we have chosen a fairly smaller sized 16 KB L1 which is kept constant across all our tests. The architecture of the cache hierarchy follows a write-allocate policy.

L1 I-cache	16kB; 64B linesize; 2-way with LRU repl.
L1 D-Cache	16kB; 64B linesize; 2-way with LRU repl.
Baseline Unified L2	256kB; 64B linesize; 4-way with LRU repl.

TABLE I: Baseline Cache configuration details

V-Way Unified L2	256kB; 64B linesize; 4-way with TDR=2
-------------------------	---------------------------------------

TABLE II: Variable-Way Cache configuration

EAF-enhanced Unified L2	256kB; 64B linesize; 4-way with $\alpha = 8$
--------------------------------	--

TABLE III: EAF Cache configuration

EAF-enhanced V-Way Unified L2	256kB; 64B linesize; 4-way with TDR=2 & $\alpha = 8$
--------------------------------------	--

TABLE IV: Combined EAF + V-Way Cache configuration

VI. RESULTS

1) *Performance of the V-Way Cache:* The performance of the V-way cache is limited by two upper bounds. A loose primary upper bound is given by the miss rate reduction of a fully associative cache of the same size. This is known as a loose upper bound because even with a single set, global reuse replacement can work better than LRU. This is because conventional LRU inserts new cache lines at the maximum priority. This implies that other blocks

in that set must be reused for the new cache line to be evicted. Authors of the V-way cache have reported that most cache lines in L2 experience little reuse. This means that even if the newly inserted cache line is of little reuse, conventional LRU will not evict it sooner. On the other hand, global reuse replacement, inserts new cache lines at minimal priority but tests other cache lines before evicting the newly inserted cache line. This means low reuse blocks are evicted sooner in the V-way cache. Another stronger secondary upper bound is given by a cache of size $TDR \times \text{Size of the V-way cache}$. Here TDR is short for Tags to Data Ratio. This is a strong upper bound because the V-way cache provides the illusion of higher associativity and capacity, but it cannot provide miss rates lower than an actual cache that really holds that data. From a baseline 4-way LRU implementation, the V-way cache gives an overall average miss-rate reduction of 21.02%. Table V below shows the access counts, miss counts and percentage miss rate reduction for the V-way cache on the SPEC 2017 benchmark suite.

2) **Performance of EAF:** As expected, highly LRU-friendly benchmarks give a higher miss rate with EAF than the vanilla LRU implementation. This behavior is observed because EAF adds an additional miss due to the block being predicted as low-reuse on the first access. Across SPEC CPU2017, the EAF outperforms LRU on average as observed from Table V, on those benchmarks whose working set size is only moderately larger than the cache, where reducing pollution lifts hit rates. Contrastingly, streaming workloads saturate the Bloom Filter rapidly, as the latest incoming lines would be demoted to LRU and thus performance falls below baseline LRU. Also tiny working sets adapt well with LRU as they fit comfortably within the LLC, where EAF's periodic filter resets and lookup overhead introduces a slight regression in performance. Thus, the EAF is greatly benefitted by workloads that exhibit high temporal reuse & yet are large enough to cause conflict misses under LRU. Fig 5 provides a visual representation of miss-rate reduction across benchmarks.

Benchmarks	L2 Accesses	L2 LRU Misses	L2 V-way Misses	L2 EAF Misses	L2 VWAY+EAF Misses	% L2 V-way Red.	% L2 EAF Red.	% L2 V-way+EAF Red.
600 PERLBENCH	2.29B	0.09B	0.018B	0.05B	0.01B	83.14	47.00	83.37
602 GCC	3.66B	0.89B	0.84B	0.86B	0.84B	5.48	3.64	5.04
603 BWAVE	12.73B	12.35B	3.58B	4.81B	2.2B	70.98	61.03	82.12
605 MCF	9.79B	7.00B	6.76B	6.11B	6.70B	3.46	12.70	4.34
607 CACTUBSSN	19.48B	1.42B	1.16B	1.89B	1.15B	17.95	-32.98	18.66
619 LBM	52.93B	34.74B	34.72B	38.64B	34.72B	0.08	-0.08	0.08
623 XALANCBMK	13.62B	0.12B	0.02B	0.05B	0.01B	84.28	56.21	84.24
625 X264	0.27B	0.06B	0.06B	0.07B	0.06B	0.38	-6.68	0.67
631 DEEPSJENG	11.60B	1.37B	1.34B	1.35B	1.34B	2.19	1.45	1.19
638 IMAGICK	2.32B	2.30B	1.92B	2.09B	2.30B	16.24	0.03	0.04
641 LEELA	4.19B	0.12B	0.08B	0.09B	0.13B	37.44	26.97	40.05
644 NAB	3.64B	0.21B	0.20B	0.20B	0.20B	3.97	4.34	4.04
648 EXCHANGE2	0.19B	0	0	0	0	14.02	-12.32	15.93
649 FOTONIK3D	10.36B	4.49B	4.48B	4.07B	4.05B	0.55	-6.79	-0.42
654 ROMS	60.27B	18.82B	17.72B	20.74B	18.35B	5.53	-10.21	2.52
657 XZ	0.84B	0.24B	0.22B	0.22B	0.23B	6.54	6.26	5.04
Average Reduction	-	-	-	-	-	21.02	9.70	21.74

TABLE V: L2 Miss Rate Breakdown and % Miss Rate Reduction Across Caches

3) **Discussion of non-performant benchmark :** Xalancbm is an XML stylesheet transformation using the Xalan-C++ XSLT processor. We notice low miss rate in the baseline, hence fewer chances for overall improvement. Xalancbm has a large instruction footprint and dynamic memory usage, but its core working set often fits within a moderately sized L2 cache. So, even with a simple LRU replacement policy, the cache captures a good portion of useful data.[16]

X264 is a Video encoding benchmark to compress raw video frames into the H.264 (MPEG-4 AVC) video format. Frame data is stored in 2D arrays (pixels), accessed in a block-wise (macroblock) manner. The encoder accesses neighboring pixels in a structured manner creating predictable memory access patterns. Thus it is cache-friendly. X264 maintains several reference frames in memory which depending on the cache size may not perfectly fit causing some extra misses.[16]

LBM belongs to the domain of computational fluid dynamics & is based on Lattice-Boltzmann Method (LBM) for simulating incompressible fluid flow. It operates over a large 3D grid with multiple fields per cell. Memory accesses are highly regular, typically iterating over entire grid slices. L1 & L2 are often ineffective — because each cell is accessed once per time step & most data isn't reused in short time frames.[16]

Deepsjeng is a deep learning benchmark evaluating AI performance in games. It is known to be highly compute intensive but has a moderate memory footprint.[16]

Nab is a nucleic acid builder benchmark. It has a small working set. It has a tiny memory footprint & hence it doesn't fill the cache enough to face conflict misses.[16]

Fotonik3d belongs to the domain of computational electromagnetics / photonics simulations. It simulates how electromagnetic (EM) waves propagate through complex 3D photonic structures. Hence it operates on a 3D grid. It faces frequent evictions, due to large arrays that don't fit in the LLC. Fotonik3d issues access to multiple arrays and buffers in parallel, which can evict useful data under LRU.[16]

Exchange2 has an extremely low L1 miss rate & hence very few L2 accesses, thus giving a very low L2 miss rate.

4) Discussion on the Results based on absolute local miss rates in the Unified L2 cache: We note that the standalone V-way performs better than or equal to the baseline LRU in all the benchmarks as seen in Table V above. The reduction in the L2 local miss rates is used here as a measure of performance. Benchmarks like perlbench, gcc, bwaves, cactubssn, xalancbmk, leela & xz that show improvements in performance in the V-Way cache benefit from the dynamic increase in associativity & the global reuse replacement policy. Benchmarks like perlbench, gcc, that show improvements in the EAF-enhanced caches benefit from the smart predictions of the EAF based on its history & thus provide proof of reduction in cache thrashing & cache pollution against the baseline LRU. Benchmarks like cactubssn, lbm, x264, fotonik3d & roms that show performance decrements in the EAF-enhanced caches follow a LRU-friendly memory access pattern incurring an extra miss on each access. Benchmarks like deepsjeng, nab & exchange2 that don't give improvements have Low L1 miss rates & hence low L2 access rates.

VII. ANALYSIS

For performance analysis a global miss rate in MPKI (Misses Per Kilo Instructions) is more informative because it ties misses directly to CPU activity. To give a system-wide picture we provide the global miss rates in Table VII. The reduction in miss-rates by the V-way cache and the EAF scale with the cache size as can be observed from

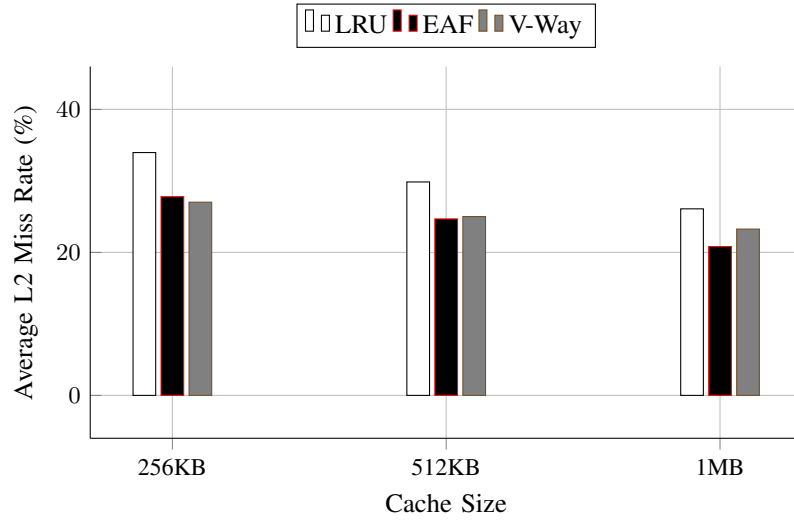


Fig. 4: Average L2 Miss Rate vs Cache Size for LRU, EAF, and V-Way

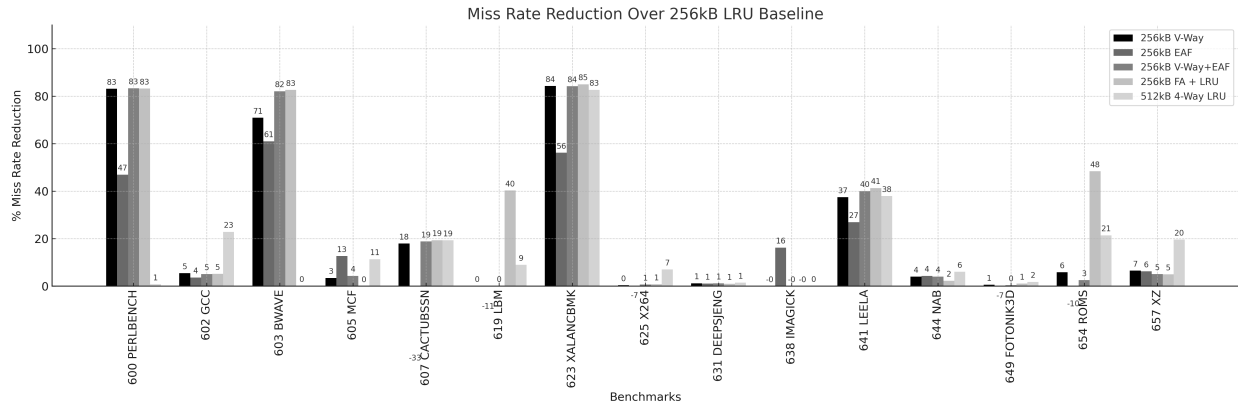


Fig. 5: Miss rate reduction relative to the baseline 256kB 4-way LRU cache across different configurations.

Fig 4. When TDR increases, the likelihood of finding an invalid-tag store increases and therefore the global reuse replacement can be invoked more frequently. This results in selection of better victims for replacement and therefore the average miss rates drop. However, saturation occurs after some time, as the replacement policy almost always finds an invalid tag store, but the data-store is full. We show TDR variation in Table VI and average variation of miss rates across benchmarks in Fig 6. While individual implementations of the V-way cache and the EAF may perform better than a baseline L2 LRU cache, the gains do not necessarily always sum up. This is because of two primary reasons. The V-way cache and EAF may both have eliminated similar conflict misses already, with less scope of improvement remaining. Further, the tag values of the evicted addresses change with the employment of the global reuse replacement. This alters the behaviour of the EAF cache dependent on the bloom filter. The alteration may or may not be beneficial for all benchmarks. Future work aims to determine when the EAF might be a better addition to the V-way cache.

Benchmarks	TDR = 1	TDR = 1.25	TDR = 1.5	TDR = 2	TDR = 3	TDR = 4
600 PERLBENCH	4.01	1.79	0.98	0.68	0.66	0.66
602 GCC	24.24	22.49	22.06	22.95	22.69	22.71
603 BWAVE	97.03	94.99	93.39	28.16	17.57	17.30
605 MCF	71.55	44.27	39.55	69.07	68.98	68.98
607 CACTUBSSN	7.28	6.51	6.27	5.99	5.91	5.91
619 LBM	65.64	76.17	77.57	65.59	65.59	65.59
623 XALANCBMK	0.85	0.37	0.25	0.13	0.14	0.14
625 X264	23.04	22.99	23.03	22.90	22.48	22.44
631 DEEPSJENG	11.75	15.97	18.12	11.62	11.61	11.61
638 IMAGICK	98.88	99.94	99.92	98.91	98.91	98.91
641 LEELA	4.93	3.48	2.83	3.23	3.16	3.12
644 NAB	6.07	5.72	5.59	5.74	5.82	5.82
648 EXCHANGE2	0.00	0.01	0.01	0.00	0.00	0.00
649 FOTONIK3D	43.38	29.45	27.27	43.14	42.94	42.91
654 ROMS	31.09	30.98	30.26	29.42	29.27	29.38
657 XZ	28.68	26.77	27.09	26.81	26.42	26.35
AVERAGE	32.40	30.12	29.64	27.15	26.38	26.36

TABLE VI: Average Miss Rate in percentage for differen TDR values

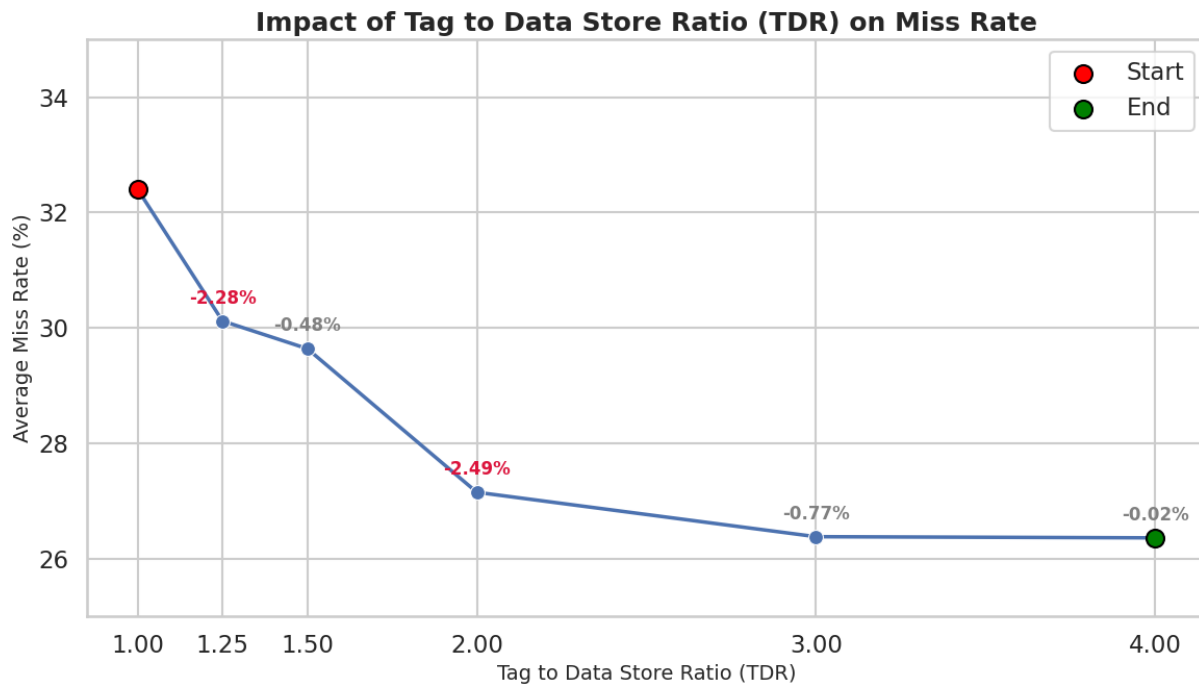


Fig. 6: Variation of Average Miss Rates with TDR (Tag To Data Ratio)

Benchmarks	Inst. Counts /1000	L2 LRU Misses	L2 V-way Misses	L2 EAF Misses	L2 VWAY+EAF Misses	MPKI LRU	MPKI V-WAY	MPKI EAF	MPKI VWAY-EAF
600 PERLBENCH	0.07B	0.09B	0.01B	0.05B	0.01B	1.29	0.22	0.69	0.21
602 GCC	0.11B	0.89B	0.84B	0.86B	0.84B	7.91	7.47	7.62	7.51
603 BWAVE	0.08B	12.35B	3.58B	4.81B	2.2B	151.62	43.99	59.06	27.10
605 MCF	0.15B	7.00B	6.76B	6.11B	6.70B	46.83	45.21	40.88	44.79
607 CACTUBSSN	0.12B	1.42B	1.16B	1.89B	1.15B	10.33	8.50	13.90	8.43
619 LBM	0.67B	34.74B	34.72B	38.64B	34.72B	51.60	51.56	57.38	51.55
623 XALANCBMK	0.29B	0.12B	0.02B	0.05B	0.01B	0.41	0.07	0.17	0.07
625 X264	0.02B	0.06B	0.06B	0.07B	0.06B	2.82	2.80	3.08	2.83
631 DEEPSJENG	0.36B	1.37B	1.34B	1.35B	1.34B	3.84	3.79	3.79	3.77
638 IMAGICK	0.28B	2.30B	2.30B	1.92B	2.30B	8.17	8.15	6.80	8.15
641 LEELA	0.39B	0.22B	0.13B	0.22B	0.13B	0.54	0.35	0.54	0.34
644 NAB	0.36B	0.21B	0.20B	0.20B	0.20B	0.59	0.57	0.57	0.57
648 EXCHANGE2	0.16B	0.00B	0.00B	0.00B	0.00B	0.06	0.05	0.06	0.05
649 FOTONIK3D	0.17B	4.49B	4.47B	4.80B	4.47B	27.01	26.87	28.65	26.90
654 ROMS	0.85B	18.82B	17.72B	20.74B	18.35B	22.09	20.78	24.31	21.62
657 XZ	0.10B	0.24B	0.22B	0.22B	0.23B	2.45	2.29	2.29	2.32

TABLE VII: Global Miss Rate in MPKI (Misses Per Kilo Instructions) for LRU, V-Way, EAF, and VWAY-EAF Caches

VIII. FUTURE WORK AND DISCUSSION

A. *Vway Cache*

Several optimizations can be applied on top of the V-way cache for optimizing power, area and performance. Experimental results show that, for most program runs, global reuse replacement is used almost exclusively. This implies that, if power is a constraint, the V-way cache need not maintain LRU counters, that consume area and whose updates increase energy consumption. A simpler replacement policy like NMRU, Random or Round Robin can be employed without significant performance penalty. Further, while the area of invalid tag stores must be incurred for the operation, these can be turned off to save on both dynamic and leakage power. The additional wake-up penalty for when these tags are made valid again is incurred in parallel with a cache miss which is already a time consuming operation. The current implementation uses reuse counters for every data-store. In an alternate implementation, a group of data-stores can share a reuse counter, and a filter can be introduced to access into the data store. Within each data-store, a simple stack can be used for replacement.

Inclusion data can be maintained in the invalid tag stores of the V-way cache. Instead of duplicating the data in the L1 and L2. The L2 only carries a tag of the block to maintain inclusion. If this tag store is now validated, on a replacement, an invalidation signal can be sent to the L1 to maintain inclusion. If the block is evicted from the L1, the data block is added to the L2, indicating a promotion. In a multicore implementation, inclusion data avoids additional L1 access if the tag in the L2 cache indicates non-inclusion.

B. *EAF*

Future work involves 2 new design approaches to enhance the EAF's performance:

1) ***Segmented-EAF***: The S-EAF works by not clearing the entire cache when the counter becomes full, rather a fraction. This is achieved by segmenting the EAF into multiple Bloom Filters, where the oldest will be cleared. This idea trades the design complexity and storage overhead with the ability to mitigate the amount of information lost due to clearing.

2) ***Decoupled-Clear-EAF***: The EAF's block reuse predictability can be enhanced by increasing visibility of the recently evicted blocks. This approach tracks twice the number of blocks in the cache. There is an additional counter that tracks the high priority insertions. Clear event happens in two cases:

- When the EAF becomes full
- When the number of high-priority blocks reaches the number of blocks in the cache

These design points sacrifice storage and increase complexity of implementation shooting up the CPI as a tradeoff to increase hit rates.

IX. CONCLUSION

An ideal cache should minimize conflict misses and avoid pollution. Pollution can be reduced by monitoring access behavior, while increased associativity helps mitigate conflicts. We require this increased associativity and history tracking without incurring exponential hardware cost. Prior work does not address thrashing and pollution simultaneously. In this work, we implement, analyze and combine the concepts of the V-Way cache and the Evicted Address Filter (EAF).

Our standalone implementations of V-Way and EAF achieved miss rate reductions of 21.02% and 9.70% on average across the SPEC CPU2017 benchmarks, respectively, compared to a baseline 256KB 4-way set-associative cache using the LRU replacement policy. Our combined implementation of both techniques resulted in a 21.74% miss rate reduction over the same baseline.

REFERENCES

- [1] V. Seshadri, O. Mutlu, M. A. Kozuch, and T. C. Mowry, "The evicted-address filter: a unified mechanism to address both cache pollution and thrashing," in *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT)*, 2012, pp. 355–366.
- [2] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Communications of the ACM*, vol. 13, no. 7, pp. 422–426, July 1970.
- [3] A. Jaleel, K. B. Theobald, S. C. Steely, Jr., and J. Emer, "High performance cache replacement using re-reference interval prediction," in *Proceedings of the 37th Annual International Symposium on Computer Architecture (ISCA)*, 2010.
- [4] T. Johnson, D. Connors, M. Merten, and W.-M. Hwu, "Run-time cache bypassing," *IEEE Transactions on Computers*, 1999.
- [5] T. Piquet, O. Rochecoste, and A. Seznec, "Exploiting single-usage for effective memory management," in *Proceedings of the ACSAC*, 2007.
- [6] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely Jr., and J. Emer, "A case for MLP-aware cache replacement," in *Proceedings of the 33rd Annual International Symposium on Computer Architecture (ISCA)*, 2006.
- [7] M. K. Qureshi and Y. N. Patt, "Utility-based cache partitioning: A low-overhead, high-performance, runtime mechanism to partition shared caches," in *Proceedings of the 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2006.
- [8] A. Jaleel, W. Hasenplaugh, M. K. Qureshi, J. S. Emer, and D. W. Wood, "High performance cache replacement using re-reference interval prediction (RRIP)," in *ISCA*, 2010.
- [9] A. Jaleel, M. K. Qureshi, and Y. N. Patt, "Set dueling: A technique to determine dynamically the better performing cache replacement policy," in *International Conference on Computer Design (ICCD)*, 2004.
- [10] J. L. Carter and M. N. Wegman, "Universal classes of hash functions," *Journal of Computer and System Sciences*, vol. 18, no. 2, pp. 143–154, 1979.
- [11] M. Mitzenmacher, "Compressed Bloom filters," *IEEE/ACM Transactions on Networking*, vol. 10, no. 5, pp. 604–612, 2002.
- [12] Hallnor, Erik G. and Reinhardt, Steven K, "A compressed memory hierarchy using an indirect index cache," *Proceedings of the 3rd Workshop on Memory Performance Issues: In Conjunction with the 31st International Symposium on Computer Architecture*, 2004, pp. 9–15
- [13] D. Sanchez and C. Kozyrakis, "The ZCache: Decoupling Ways and Associativity," *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture, Atlanta, GA, USA, 2010*, pp. 187–198
- [14] M. K. Qureshi, D. Thompson and Y. N. Patt, "The V-Way cache: demand-based associativity via global replacement," *32nd International Symposium on Computer Architecture (ISCA'05), Madison, WI, USA, 2005*, pp. 544–555

- [15] Intel Corporation, *Pin - A Dynamic Binary Instrumentation Tool*. <https://www.intel.com/content/www/us/en/developer/tools/pin-tool/overview.html>, Accessed: 2025-05-04.
- [16] Navarro-Torres A, Alastruey-Benedé J, Ibáñez-Marín P, Viñals-Yúfera V. *Memory hierarchy characterization of SPEC CPU2006 and SPEC CPU2017 on the Intel Xeon Skylake-SP*. *PLoS One*. 2019 Aug 1

APPENDIX A

EAF CODE IMPLEMENTATION IN INTEL PIN

Structure of the EAF constructor

```
struct EAF {
    UINT32 maxLines;    // cache lines tracked
    UINT32 counter;     // #inserts since last reset
    UINT32 bits;        // bits = alpha * maxLines
    std::vector<UINT8> bf; // bit vector
    std::array<std::vector<ADDRINT>, 3> masks; // H3 rows
};
```

Size (in bits) = $\alpha \times \text{maxLines}$ ($\alpha = 8$; $\approx 1.6\%$ space overhead for a 8-way LLC). Counter tracks how full the filter is; clearing occurs automatically when the counter reaches maxLines.

Hash functions (H3)

```
inline UINT32 h(int i, ADDRINT x) const {
    UINT32 idx = 0;
    for (UINT32 b = 0; b < logSize; ++b)
        if (__builtin_parityll(masks[i][b] & x))
            idx |= (1u << b);
    return (idx >= bits) ? idx % bits : idx;
}
```

The parity of each row $\oplus$ tag pair yields one bit of the index to the Bloom Filter.

Helper Functions

```
bool test(ADDRINT tag) const {
    return testbit(h1(tag)) && testbit(h2(tag)) && testbit(h3(tag));
}

void insert(ADDRINT tag) {
    setbit(h1(tag)); setbit(h2(tag)); setbit(h3(tag));
    if (++counter == maxLines // full?
        clear(); // reset BF & counter
}
```

insert() sets three bits and increments the counter. Auto-reset: when the number of inserted evictions equals cache-lines, the Bloom filter is cleared; this bounds the false-positive rate to $\approx 4\text{--}6\%$. *test()* is used only on an LLC miss path.

LLC miss-path

```
bool highReuse = ul3_eaf.test(tag);           // reuse prediction
UINT32 newAge = highReuse ? maxAge            // insertion priority
                : ((rand() & 0x3F) ? 0 : maxAge); // BIP
CACHE_TAG victim = set.Replace(tag, newAge);  // replace line
ul3_eaf.insert(victim);                      // remember victim
```

The filter is queried and updated in a small wrapper around the baseline LLC replacement function.

Occupancy

```
counter           // incremented on every insert
ul3_eaf.clear()   // sets counter  $\leftarrow 0$  (called automatically)
```

Occupancy is the only statistic we needed to track in EAF's activity. It is sampled at the end of each run to compute filter utilisation.

APPENDIX B

V-WAY CODE IMPLEMENTATION IN INTEL PIN

Find function in the VWAY cache

```

UINT32 Find(CACHE_TAG tag, UINT32 set_index)
{
    bool result = true;
    if (set_data[set_index].current_associativity > 0)
    {
        for (UINT32 index = 0;
             index <= (set_data[set_index].max_associativity-1); index++)
        {
            if (tag_stores[set_index][index].isValid() &&
                tag_stores[set_index][index].get_tag() == tag)
            {
                for (UINT32 index2 = 0;
                     index2 <= (set_data[set_index].max_associativity-1);
                     index2++)
                {
                    if (index2 != index &&
                        tag_stores[set_index][index2].isValid() &&
                        tag_stores[set_index][index2].get_count() >
                        tag_stores[set_index][index].get_count())
                    {
                        tag_stores[set_index][index2].decrement_count();
                    }
                }
                // maximize the counter of found index
                tag_stores[set_index][index].
                    set_count(set_data[set_index].
                               current_associativity-1);
                data_array._data_store[tag_stores[set_index][index].
                                       get_FPTR()].increment_reuse_counter();
                goto end;
            }
        }
        result = false;
    end:
    return result;
}

```

The Find() function is called for all cache accesses. If the current associativity of the cache set zero, then the function exits. Else it runs a loop on all the ways within the set. Upon accessing a valid way, it checks for a tag match. On a successful tag-match, it decrements the LRU counters of the other valid ways with counter value greater than the way that matched the tag. Then it sets the LRU counter of the way accessed to current_associativity -1. Lastly, the reuse counter of the data store is incremented.

If-case in the Replace() function

```
if(set_data[set_index].current_associativity <
    set_data[set_index].max_associativity)
```

The Replace() function is called only if there is a cache miss i.e. the Find() doesn't fail to get a tag match. The if-case within the Replace() function is a call to the global reuse replacement.

Step-1: If current set associativity is less than max associativity; increase the associativity

```
// invalid way gives the victim index
for (UINT32 index = 0; index <= (set_data[set_index].max_associativity-1);
    index++)
{
    if(!tag_stores[set_index][index].isValid())
    {
        victim_index = index;
        break;
    }
}
// update the corresponding tag store
tag_stores[set_index][victim_index].set_tag(tag);
tag_stores[set_index][victim_index].toggleValid();
tag_stores[set_index][victim_index].set_count(set_data[set_index].
current_associativity);
//demand based growth in associativity
set_data[set_index].current_associativity++;
```

In case the target set has invalid tag stores, we determine that the associativity can be increased. A previously invalid tag is selected to hold the new tag. This tag store then using the methods of the *CACHE_METADATA* class is made to store the new tag value, validated and its counter set to maximum value among all tag-stores.

Step-2: Determine Data Victim

```
while(data_array._data_store[data_array.PTR].get_reuse_counter() != 0)
{
    data_array._data_store[data_array.PTR].decrement_reuse_counter();
    data_array.PTR++;
    // reset PTR on reaching the end of the data store
    if(data_array.PTR == (data_array.data_store_size))
        data_array.PTR = 0;
} // PTR points to the data victim

// Map new tag store to its data block
tag_stores[set_index][victim_index].set_FPTR(data_array.PTR);
```

We then use the global reuse replacement policy to find the data_victim, a data_store with reuse counter value zero. The pointer moves circularly till it finds the victim. Once found the *FPTR* of the previously found tag_store of the newly validated tag is updated to hold this *PTR* value.

Step-3: Update the Reverse Pointer

```

UINT64 data_victim_rptr = data_array._data_store[data_array.PTR].get_RPTR();
UINT64 new_rptr = set_index * set_data[set_index].max_associativity +
                  victim_index;
data_array._data_store[data_array.PTR].set_RPTR(new_rptr);

data_array.PTR++; //Give this new block some time to be reused.
if(data_array.PTR == (data_array.data_store_size))
    data_array.PTR = 0;

```

The previous *RPTR* of the *data_victim* is stored to obtain the *tag_victim* belonging to the set that will lose associativity. In the same way a 2-D array is accessed, we get the global index of the validated tag and assign this to the *RPTR* of the *data_store*

Step-4: Invalidate the Tag Victim

```

if(data_victim_rptr != UINT64(-1))
{
    UINT32 victim_set_index =
        data_victim_rptr/set_data[set_index].max_associativity;
    UINT32 victim_way = data_victim_rptr%set_data[set_index].max_associativity;
    tag_stores[victim_set_index][victim_way].toggleValid();
    tag_stores[victim_set_index][victim_way].set_tag(CACHE_TAG(0));

    if(set_data[victim_set_index].current_associativity > 0)
    {
        // update the counters in the victim set
        for(UINT32 index=0;
            index < set_data[victim_set_index].max_associativity; index++)
        {
            // update only if counter value is greater than the victim &
            the way is valid
            if(tag_stores[victim_set_index][index].isValid() &&
                tag_stores[victim_set_index][victim_way].get_count() <
                tag_stores[victim_set_index][index].get_count())
            {
                tag_stores[victim_set_index][index].decrement_count();
            }
        }
        tag_stores[victim_set_index][victim_way].
            set_count(set_data[victim_set_index].current_associativity-1);

        set_data[victim_set_index].current_associativity--;
    }
}

```

Unless the new *data_store* was already invalid, we used the stored *RPTR* to obtain the *tag_victim* and invalidate it.

The LRU counters are then updated. Since this set has one more invalid tag its associativity is decreased.

Else-case in the Replace() function

```

else
{
    for(UINT32 index = (set_data[set_index].max_associativity-1);
        index >= 0 && index <= (set_data[set_index].max_associativity-1);
        index--)
    {
        if(tag_stores[set_index][index].get_count() == 0)
        {
            victim_index = index;
            break;
        }
    }

    // decrement the value of the other counters
    for(UINT32 index = set_data[set_index].max_associativity-1;
        index >= 0 && index <= (set_data[set_index].max_associativity-1);
        index--)
    {
        if(index != victim_index &&
            tag_stores[set_index][index].get_count() > 0)
        {
            tag_stores[set_index][index].decrement_count();
        }
    }

    tag_stores[set_index][victim_index].set_tag(tag);
    tag_stores[set_index][victim_index].
    set_count(set_data[set_index].max_associativity-1);
    // tag_stores[set_index][victim_index].toggleValid();
    data_array._data_store[tag_stores[set_index][victim_index].get_FPTR()].
    set_reuse_counter(0);
    data_array.PTR = tag_stores[set_index][victim_index].get_FPTR()+1;
    if(data_array.PTR == (data_array.data_store_size))
        data_array.PTR = 0;
}

```

The else-case in the Replace() function is called when the associativity of the current set has reached its maximum value. The V-WAY cache then resorts to standard LRU policy.